

MMML cli tutorial

This walkthrough follows the numbered scripts in `mmml_tutorial/cli/`: build a small system with PyCHARMM, run GPU DFT and sampling, label data, split into training NPZs, train PhysNet (optional), and train the joint PhysNet+DCMNet model with ESP grids. Default residue is BENZ (shared.source); override with `RES=TIP3` (or another residue) before sourcing if you want a different chemistry.

Each step below links a **figure** (molecule, plot, or schematic) and a **log excerpt** from the checked-in `*.log` files in `cli/`, so a prepared tutorial room can show expected console output without re-running long jobs.

For a flat command index, see `cli_cheatsheet.typ`.



MMML

<https://github.com/EricBoittier/mmml>



mmml_tutorial

https://github.com/EricBoittier/mmml_tutorial

Setup: uv, MMML, PyCHARMM, and Packmol

Install `uv` once per machine. The standalone installer is simplest; `pip` or `conda` are fine when those tools are already managed on the machine:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv --version
```

Alternatives

```
python -m pip install --user uv
conda install -c conda-forge uv
```

Clone and install MMML into a project-managed environment. The package currently targets Python 3.13. Prefer the full extra for tutorial machines:

```
git clone https://github.com/EricBoittier/mmml.git
cd mmml
uv python install 3.13
uv sync --extra all
uv run mmml --help
```

Choose a narrower full extra only when the hardware requires it:

```
# CPU-only machines or laptops
uv sync --extra all-cpu
```

```
# CUDA 12 nodes
uv sync --extra all-cuda12
```

For editable development from an already active environment, the equivalent install is:

```
uv pip install -e ".[all]"
```

Compile CHARMM for PyCHARMM

MMML's system-building and hybrid-MD paths import PyCHARMM and expect CHARMM to be built as a shared library. From a licensed CHARMM source tree:

```
cd /path/to/charmm
./configure --as-library
make -j "$(nproc)"
```

Then point MMML at that build with a `CHARMMSETUP` file in the MMML repository root:

```
cat > CHARMMSETUP <<'EOF'  
CHARMM_HOME=/path/to/charmm  
CHARMM_LIB_DIR=/path/to/charmm/build/cmake  
EOF
```

Quick check:

```
uv run python - <<'PY'  
from mmml.interfaces.pycharmmInterface.import_pycharmm import CHARMM_HOME, CHARMM_LIB_DIR  
print("CHARMM_HOME", CHARMM_HOME)  
print("CHARMM_LIB_DIR", CHARMM_LIB_DIR)  
PY
```

If a cluster build segfaults during CHARMM energy display calls, run tutorial commands with SKIP_CHARMM_ENERGY_SHOW=1 or pass --skip-energy-show where the CLI exposes it.

Install or compile Packmol

mmml make-box shells out to the packmol executable. Install the binary with conda when possible:

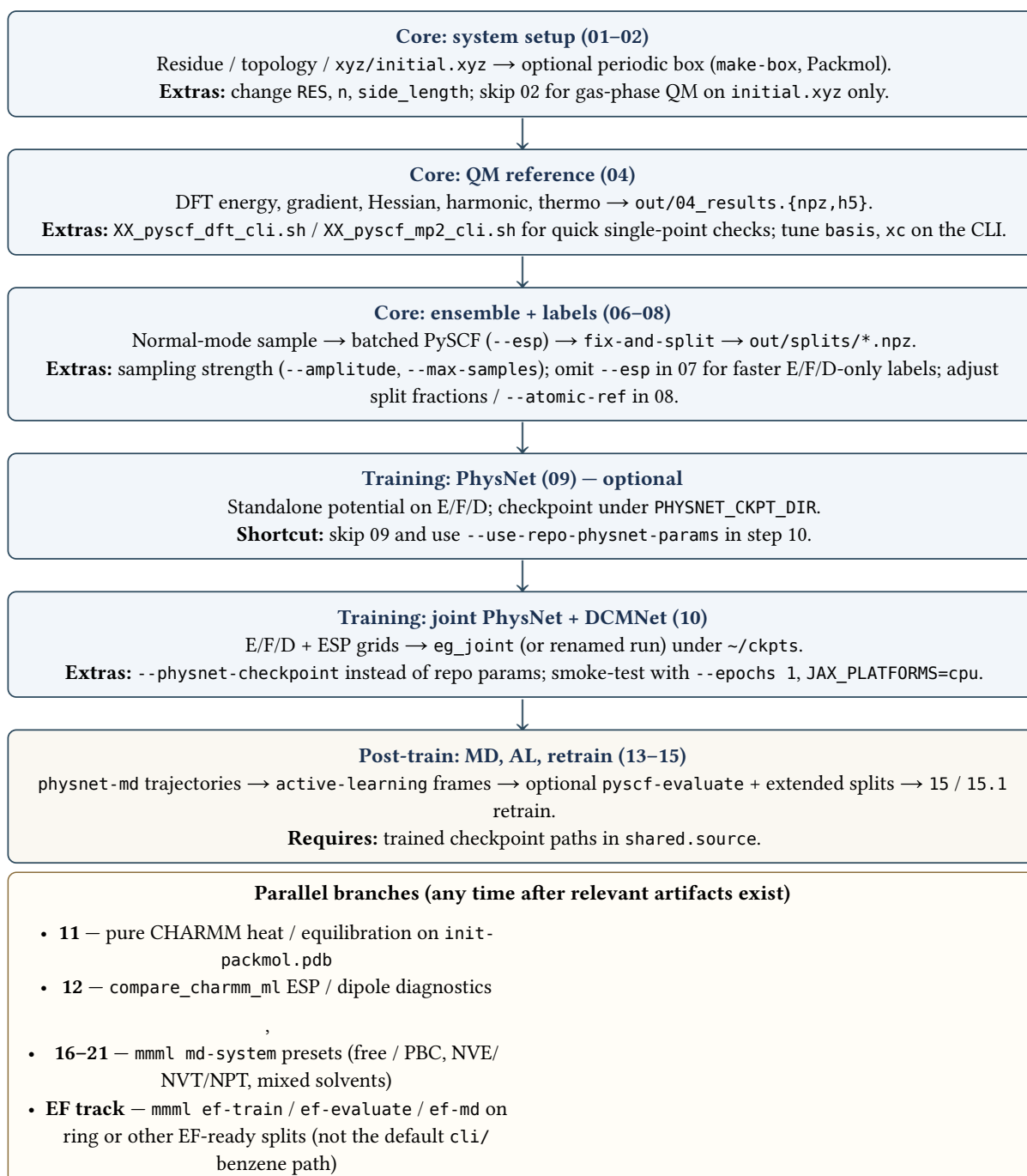
```
conda install -c conda-forge packmol  
command -v packmol
```

If conda is not available, compile Packmol from source and put the executable on PATH:

```
git clone https://github.com/m3g/packmol.git  
cd packmol  
make  
export PATH="$PWD:$PATH"  
command -v packmol
```

Tutorial map: core path, options, and extras

The figure below is the **contract** for the rest of the document: a linear **core** pipeline (stacked boxes) you can trim for demos, plus **optional branches** (bottom panel) for classical MM, diagnostics, neural MD, active learning, preset md-system workloads, and equivariant field (EF) models.



Listing 1: End-to-end **tutorial coverage** (center column) versus **extra functionality** you can demonstrate with the same repo (branches). Numbers refer to scripts in mmml_tutorial/cli/.

How to choose options at each stage

- **Residue and box (01-02).** Use shared.source or RES=... for chemistry. Need a condensed phase or interface? Increase --n and box side_length, or add solvent flags if your make-box invocation supports them. Gas-phase dataset: run 01 only and point step 04 at xyz/initial.xyz.
- **Reference QM (04).** The tutorial script requests Hessian + harmonic for normal-mode sampling. For a **cheap** sanity check, use the XX_* scripts or a minimal pyscf-dft run without --hessian. Production datasets usually keep harmonic data for 06.
- **Sampling (06).** --max-samples trades cost (step 07) against diversity. Multiple --amplitudes are supported by the CLI if you customize the script.

- **Batch evaluation (07).** `--esp` adds cost and ESP grid tensors required for DCMNet. For pure PhysNet-on-EF training without ESP, you can drop `--esp` and adjust step 08 / 10 inputs accordingly.
- **Splits (08).** `fix-and-split` chooses train/valid/test fractions and optional atomic energy references. For **active learning**, merge new evaluated NPZs into extended directories (`splits_extended*`) as in steps 14–15.
- **PhysNet vs joint (09–10).** If the portable repo PhysNet matches your stoichiometry, prefer `--use-repo-physnet-params` and skip long 09 runs in the classroom. Use `--physnet-checkpoint` when you need your own step-09 run.
- **After training.** Step 13 drives exploratory MD; step 14 subsamples frames for re-QM. Steps 16–21 showcase `md-system` automation (cutoff figures in the tutorial assets). The EF subsection documents a **different** model family on its own NPZ layout.

Figures and log excerpts

- Regenerate PNGs under `assets/cli/` with `uv run python generate_tutorial_assets.py` from `mmml_tutorial/cli/`.
- Log snippets are illustrative; your exact timings and energies will differ.

Recent CLI updates

- Joint training: `--use-repo-physnet-params` bootstraps PhysNet from portable repo JSON (no fresh PhysNet run required for a DCMNet demo).
- EF training: `--rot-augment`, `--rot-perturbation`.
- Checkpoints: `mmml extract-checkpoint-metrics <run_dir> -o metrics.png --log-loss`.

What this tutorial covers

See **Tutorial map** (above) for the full pipeline and branches. In short: scripts 01–08 build the ML dataset; 09–10 train; 11–21 and EF commands are optional extensions using the same CLI.

Live tutorial strategy

1. Walk through steps 01–02 and 04 conceptually (show figures + logs).
2. Run or show 06–07 only if GPUs are ready; otherwise use precomputed out/.
3. For DCMNet, either run 08 then 10, or supply prepared `out/splits/*.npz`.
4. Smoke-test joint training: `--epochs 1, --ckpt-dir /tmp/mmml_ckpts, JAX_PLATFORMS=cpu` on laptops.

Quickest joint-training demo (after splits exist):

```
cd mmml_tutorial/cli
bash 10_physnet_dcmnet_train_cli.sh
```

Environment checklist

- Work from `mmml_tutorial/cli` (paths below are relative to that directory).
- Install `uv`, run `uv python install 3.13`, and prefer `uv sync --extra all` from the MMML repo.
- PyCHARMM + Packmol for steps 01–02; PySCF/GPU stack for 04 and 07.
- Compile CHARMM with `./configure --as-library`, then set `CHARMM_HOME` and `CHARMM_LIB_DIR` in `CHARMMSETUP`.
- Install Packmol with `conda install -c conda-forge packmol` or compile it from source and add it to `PATH`.
- `JAX_PLATFORMS=cpu` if CUDA init breaks before `argparse`.

Directory map (`mmml_tutorial/cli/`)

- `shared.source` — RES, PhysNet trainer path, checkpoint dirs, NPZ paths, MD defaults.
- `01_make_res_cli.sh` — residue/topology + `xyz/initial.xyz`.
- `02_make_box_cli.sh` — Packmol box `pdb/init-packmol.pdb`.
- `04_pyscf_dft_cli_full.sh` — DFT + Hessian + harmonic + thermo → `out/04_results.{npz,h5}`.
- `06_normal_mode_sample_cli.sh` — `out/06_sampled.npz`.
- `07_pyscf_evaluate_cli.sh` — batched QM + ESP → `out/07_evaluated*.npz`.
- `08_fix_and_split_cli.sh` — splits under `out/splits/`.

- 09_physnet_train_cli.sh — PhysNet (trainer script from shared.source).
- 10_physnet_dcmnet_train_cli.sh — joint PhysNet+DCMNet → ~/ckpts/eg_joint/...
- 11_run_pycharmm_cli.sh — classical heat/equilibration.
- 12_charmm_esp_dipole_comparison.sh — CHARMM vs ML diagnostics.
- 13_physnet_md.sh — PhysNet MD trajectories.
- 14_active_learning.sh — frames for re-labeling.
- 15_physnet_retrain_cli.sh, 15.1_physnet_retrain_cli.sh — continued training.
- 16_md_10mer_free_nve.sh ... 20_md_10mer_pbc_npt.sh — mmml md-system presets.
- 21_md_system_meoh_tip3_1to1.sh — mixed composition example.
- run_full_training.sh — 06→10 automation with shortened epochs in-repo.

Data flow

```
xyz/initial.xyz
-> out/04_results.{npz,h5}
-> out/06_sampled.npz
-> out/07_evaluated*.npz
-> out/splits/{energies_forces_dipoles,grids_esp}_{train,valid,test}.npz
-> ~/ckpts/<PHYSNET_NAME>/ (step 09, optional)
-> ~/ckpts/eg_joint/ (step 10, Orbax run id may include a UUID suffix)
```

Recommended run order

1. 01_make_res_cli.sh
2. 02_make_box_cli.sh
3. 04_pyscf_dft_cli_full.sh
4. 06_normal_mode_sample_cli.sh
5. 07_pyscf_evaluate_cli.sh
6. 08_fix_and_split_cli.sh
7. 09_physnet_train_cli.sh (**optional if using repo PhysNet params in step 10**)
8. 10_physnet_dcmnet_train_cli.sh
9. 13_physnet_md.sh, 14_active_learning.sh, 15*.sh as needed
10. 16–21 MD presets for md-system exploration

Side branches: 11_run_pycharmm_cli.sh, 12_charmm_esp_dipole_comparison.sh.

Step-by-step notes

Step 01: Make residue

Purpose: CHARMM residue/topology and xyz/initial.xyz for the chosen RES.

```
cd mmml_tutorial/cli
bash 01_make_res_cli.sh
```

xyz/initial.xyz

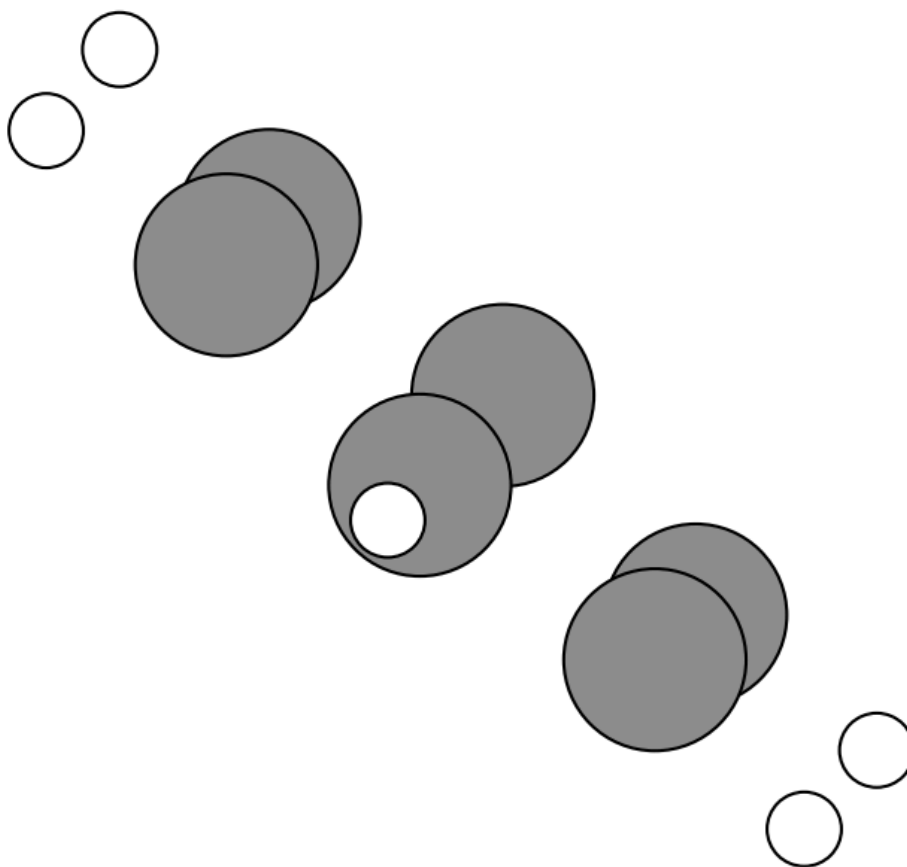


Figure 1: Study system (BENZ by default): ASE plot_atoms on xyz/initial.xyz (asset script falls back to built-in C6H6 if missing).

Main outputs: pdb/initial.pdb, psf/initial.psf, xyz/initial.xyz, out/01_last_command.txt.

```
=== 01: make_res (CLI) ===  
Command: mmml make-res --res BENZ --skip-energy-show  
...  
NORMAL TERMINATION BY NORMAL STOP  
Saved command: out/01_last_command.txt
```

Listing 2: Tail of mmml_tutorial/cli/01.log (CHARMM output abbreviated).

Step 02: Build box

Purpose: Pack copies into a cubic cell for classical equilibration or visualization.

`bash 02_make_box_cli.sh`

init-packmol.pdb (24 of 24 atoms)

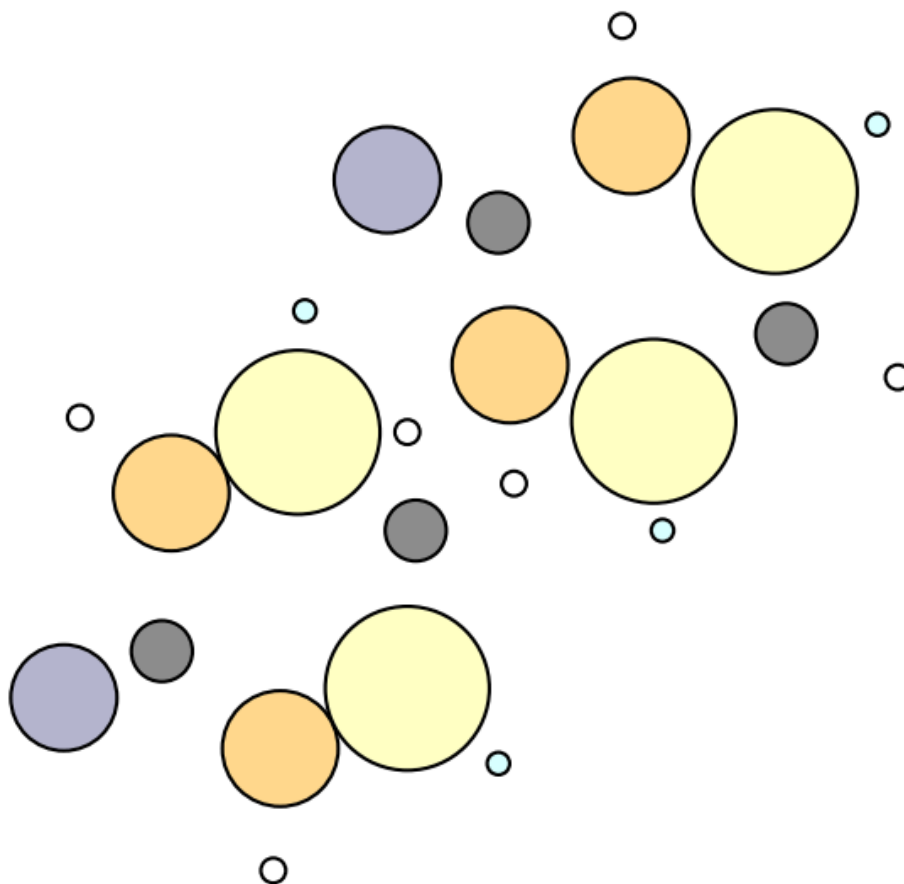


Figure 2: ASE plot_atoms on a subset of pdb/init-packmol.pdb (large cells are truncated for clarity).

```
=== 02: make_box (CLI) ===  
Command: mmml make-box --res BENZ --n 2 --side_length 25.0  
...  
Saved command: out/02_last_command.txt
```

Listing 3: Excerpt from cli/02.log.

Step 04: Full DFT with PySCF

Purpose: reference energy, gradient, Hessian, harmonic analysis, and thermo on xyz/initial.xyz.

`bash 04_pyscf_dft_cli_full.sh`

Step 04 — QM reference

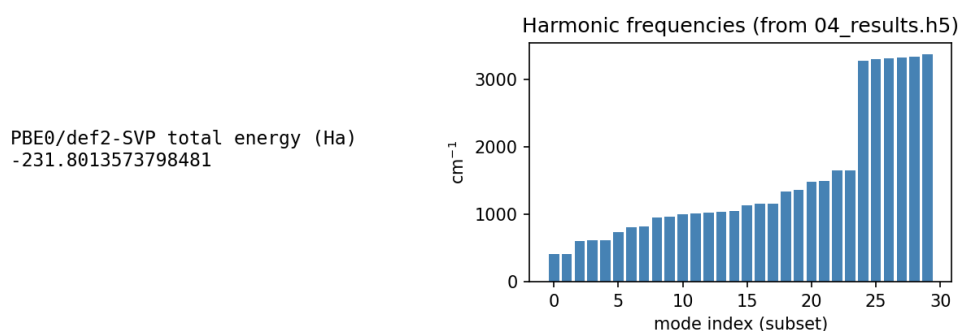


Figure 3: DFT total energy (from 04.log) and harmonic frequencies when out/04_results.h5 is present.

Main outputs: out/04_results.npz, out/04_results.h5, pyscf.log.

```
=== 04: DFT full (energy, gradient, hessian, harmonic, thermo) ===
Command: uv run mmml pyscf-dft --mol xyz/initial.xyz --energy --gradient --hessian --harmonic --thermo
--output out/04_results
...
total energy = -231.8013573798481
...
Results saved to out/04_results.npz (ML keys) and out/04_results.h5 (all arrays)
Elapsed: 16.68 s
```

Listing 4: Excerpt from cli/04.log.

Step 06: Normal-mode sampling

Purpose: build a local geometry ensemble from the harmonic model.

`bash 06_normal_mode_sample_cli.sh`

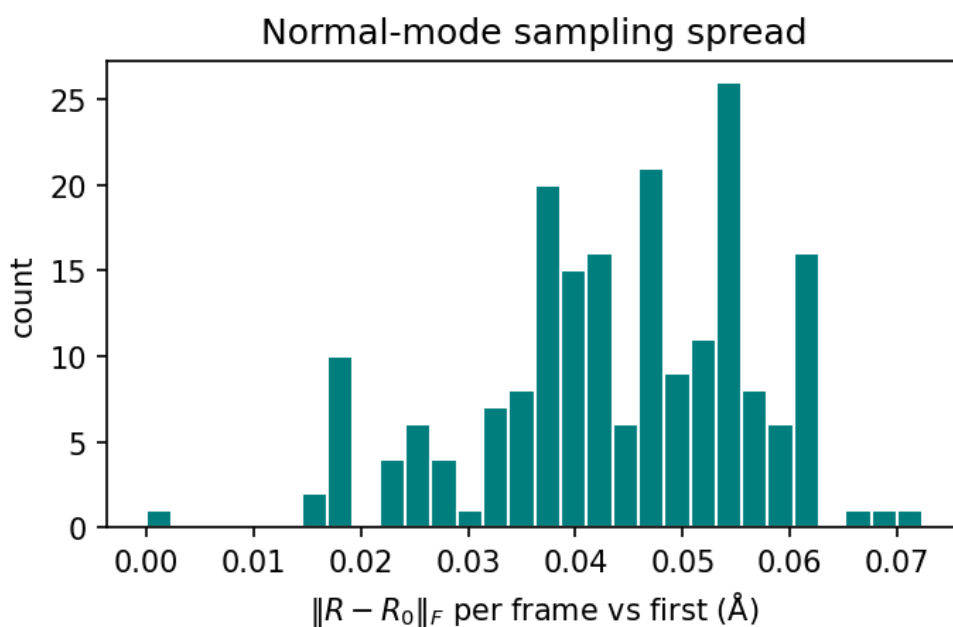


Figure 4: Distribution of displacements relative to the first sampled frame (out/06_sampled.npz).


```

=== 06: Normal mode sampling ===
Command: mmml normal-mode-sample -i out/04_results.h5 -o out/06_sampled.npz --amplitude 0.1 --max-samples 200
Saved 200 geometries to out/06_sampled.npz
Elapsed: 0.03 s

```

Listing 5: Excerpt from cli/06.log.

Step 07: Evaluate sampled geometries

Purpose: batch QM labels and ESP grids for ML.

`bash 07_pyscf_evaluate_cli.sh`

If out/07_evaluated.npz already exists, the CLI may write out/07_evaluated_2.npz (see log). Step 08 picks the newest out/07_evaluated*.npz.

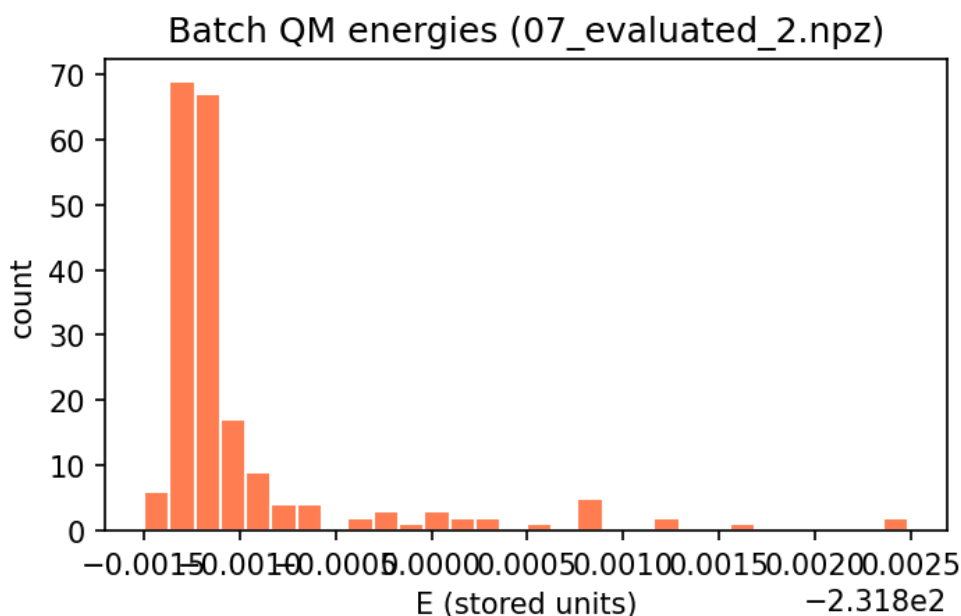


Figure 5: Histogram of total energies across the evaluated batch (out/07_evaluated*.npz).

```

=== 07: pyscf-evaluate (energy, forces, dipoles, ESP) ===
Command: uv run mmml pyscf-evaluate -i out/06_sampled.npz -o out/07_evaluated.npz --esp
Evaluating 200 geometries with pyscf-dft (GPU)...
Output exists, using out/07_evaluated_2.npz instead of out/07_evaluated.npz
Saved to out/07_evaluated_2.npz
  R: (200, 12, 3), E: (200,), F: (200, 12, 3)
Elapsed: 502.56 s

```

Listing 6: Excerpt from cli/07.log.

Step 08: Fix units and split datasets

Purpose: atomic references, unit cleanup, deterministic train/valid/test NPZs.

`bash 08_fix_and_split_cli.sh`

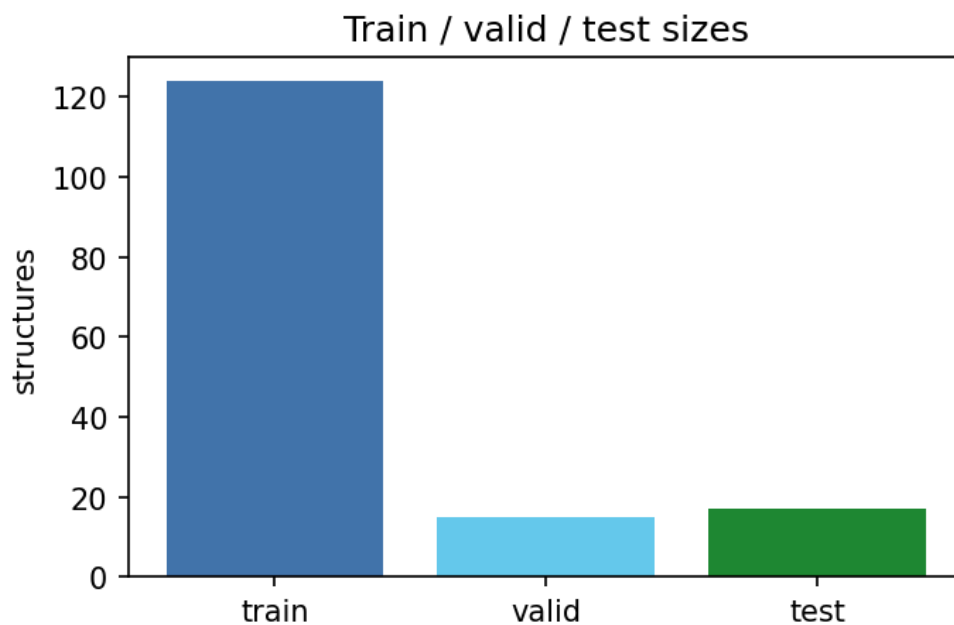


Figure 6: Structure counts in out/splits/ after fix-and-split.

```
=== 08: fix-and-split (train/valid/test) ===
Command: mmml fix-and-split --efd out/07_evaluated_2.npz --output-dir out/splits
```

Listing 7: The shell script passes the newest matching out/07_evaluated NPZ (see 08_fix_and_split_cli.sh).

Typical outputs:

- out/splits/energies_forces_dipoles_{train,valid,test}.npz
- out/splits/grids_esp_{train,valid,test}.npz

Step 09: Train PhysNet

Purpose: baseline potential on E/F/D (charges + ZBL in the bundled trainer invocation).

`bash 09_physnet_train_cli.sh`

Checkpoints land under "\$PHYSNET_CKPT_DIR/\$PHYSNET_NAME/" (see shared.source).

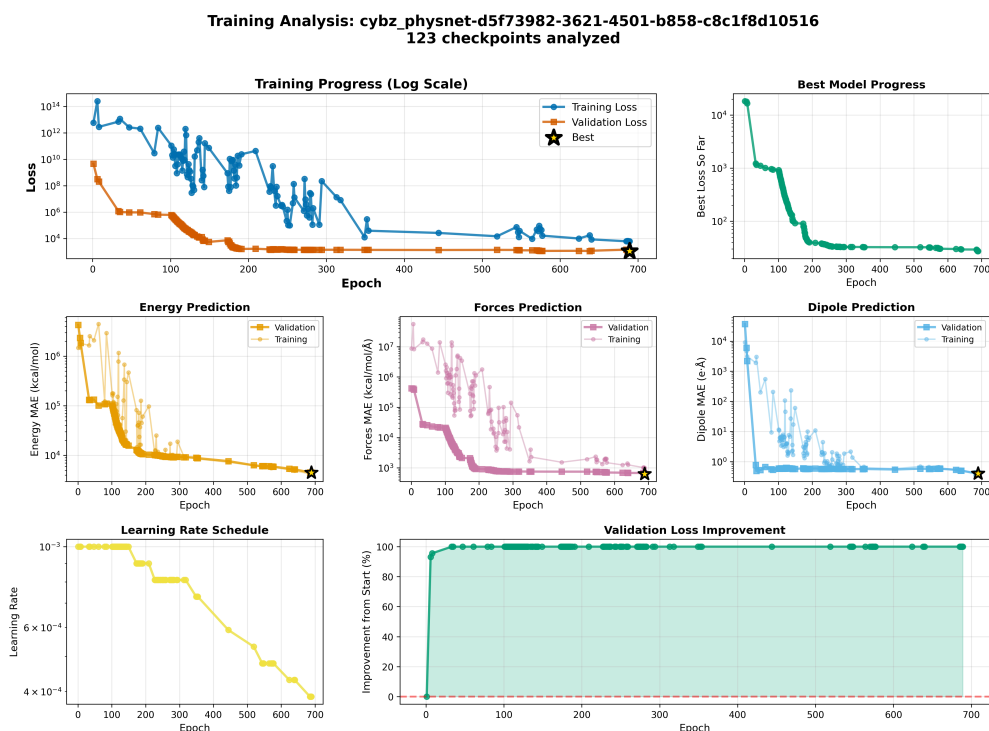


Figure 7: Orbx metrics from mmml extract-checkpoint-metrics when a PhysNet run directory is available; otherwise regenerate assets with placeholders.

```
mmml extract-checkpoint-metrics "$PHYSNET_CKPT_DIR/$PHYSNET_NAME" \
-o typst_docs/tutorial/assets/cli/step09_training_metrics.png --log-loss
```

(Adjust the output path if you run the command from another working directory.)

Step 10: Joint PhysNet + DCMNet training

Purpose: co-train on energies, forces, dipoles, and ESP grids.

```
bash 10_physnet_dcmnet_train_cli.sh
```

Equivalent command (from the script):

```
python -m mmml.cli.misc.train_joint \
--train-efd out/splits/energies_forces_dipoles_train.npz \
--train-esp out/splits/grids_esp_train.npz \
--valid-efd out/splits/energies_forces_dipoles_valid.npz \
--valid-esp out/splits/grids_esp_valid.npz \
--use-repo-physnet-params \
--epochs 1000 \
--batch-size 1 \
--name eg_joint \
--ckpt-dir ~/ckpts --plot-results --plot-freq 0
```

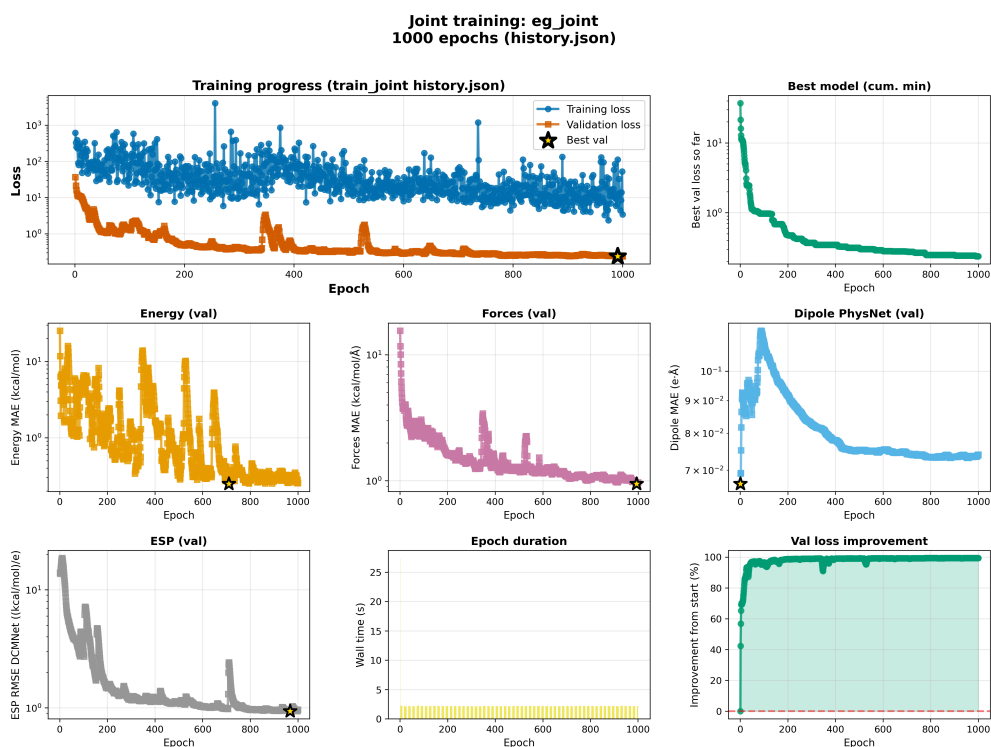


Figure 8: Joint training metrics; generate with `extract-checkpoint-metrics` on `~/ckpts/eg_joint*`.

Notes:

- Use only one of `--use-repo-physnet-params`, `--physnet-checkpoint`, or `--restart`.
- Smoke test: `--epochs 1, --ckpt-dir /tmp/mmml_ckpts`.

EF training: rotational augmentation example

```
mmml ef-train \
  --train-npz out/splits_ef0_ring/energies_forces_dipoles_train.npz \
  --valid-npz out/splits_ef0_ring/energies_forces_dipoles_valid.npz \
  --test-npz out/splits_ef0_ring/energies_forces_dipoles_test.npz \
  --rot-augment --rot-perturbation 1.0 \
  --clip_norm 1000 \
  --output-dir ~/ckpts/ring_ef_ptT_run2 \
  --num_iterations 2 --max_degree 2
```

(EF ring dataset paths are examples; adapt to your splits.)

Step 11: Pure CHARMM equilibration

```
bash 11_run_pycharmm_cli.sh
```

init-packmol.pdb (24 of 24 atoms)

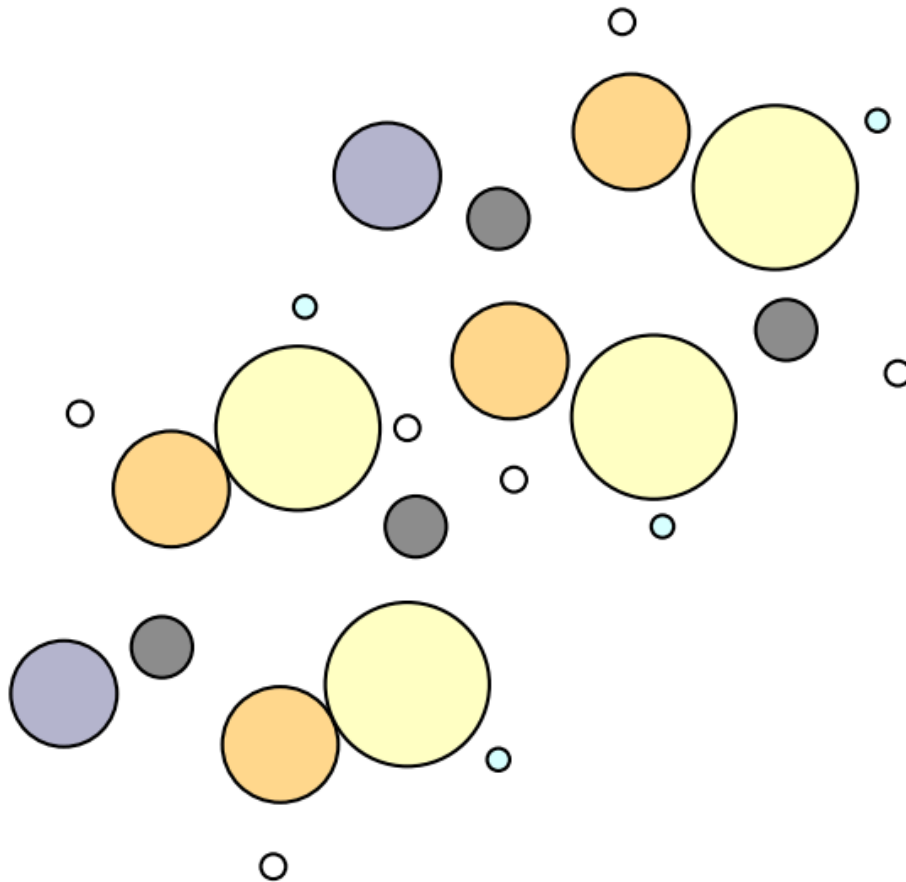


Figure 9: Same packed cell as step 02: this is the geometry family fed to run-pycharmm before heat/equilibration outputs are written.

Outputs: `pdb/{heat,equi}.pdb`, `dcd/{heat,equi}.dcd`, `res/{heat,equi}.res`.

Step 12: CHARMM vs ML comparison

`bash 12_charmm_esp_dipole_comparison.sh`

The script runs `compare_charmm_ml` against `~/ckpts/eg_joint` and `out/splits/*_test.npz`; edit the checkpoint path if your joint run uses another directory.

After compare_charmm_ml:
inspect parity plots / HDF5 in out-dir

Figure 10: Placeholder: replace by parity or GUI screenshots after you run compare_charmm_ml and regenerate assets if desired.

Step 13: PhysNet MD sampling

```
bash 13_physnet_md.sh
```

Run 13_physnet_md.sh
→ out/physnet_md/*.xyz

Figure 11: ASE plot_atoms on the last frame of ASE or JAX-MD xyz under out/physnet_md/ when trajectories exist.

Step 14: Active learning extraction

```
bash 14_active_learning.sh
```

Run 14_active_learning.sh
→ e.g. out/activate_learning.npz

Figure 12: Spread of geometries in the active-learning NPZ (out/activate_learning.npz by default in shared.source).

Suggested follow-up:

```
mmml pyscf-evaluate -i "$ACTIVE_LEARNING_OUT" -o out/md_evaluated.npz --esp
mmml fix-and-split --efd "$PHYSNET_TRAIN_NPZ" out/md_evaluated.npz -o out/splits_extended
```

Step 15 / 15.1: Retraining on extended splits

Continue PhysNet training with --restart pointing at your previous run; see 15_physnet_retrain_cli.sh and 15.1_physnet_retrain_cli.sh for one or multiple extension directories (splits_extended/).

xyz/initial.xyz

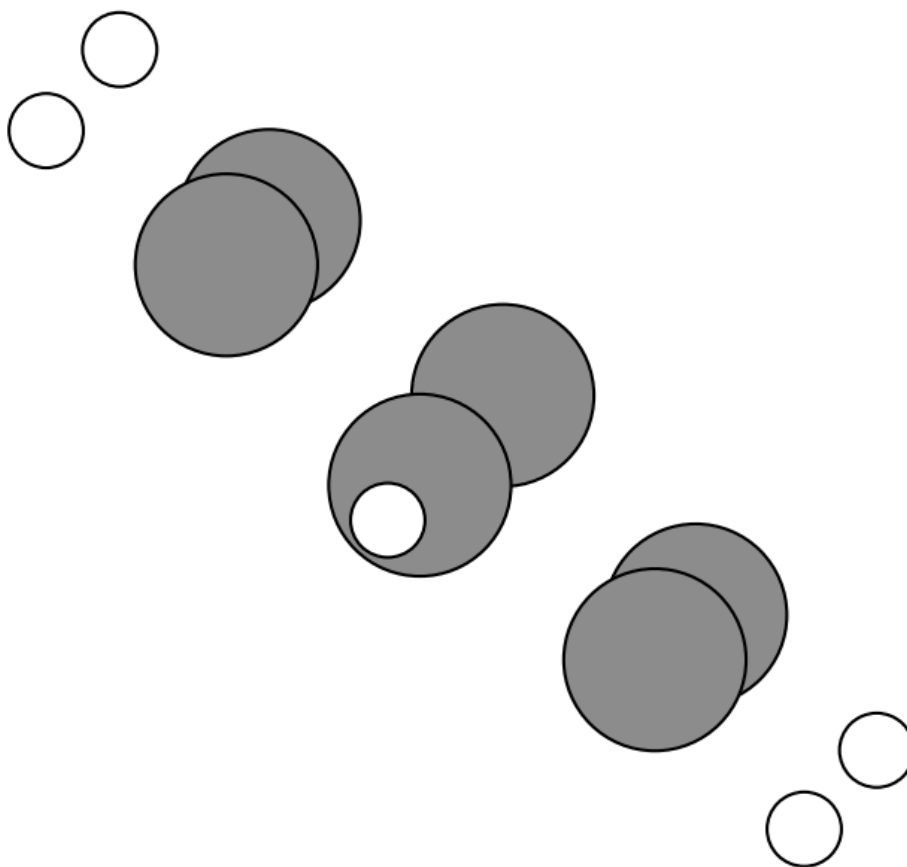


Figure 13: Chemistry reminder: extended datasets still target the same RES / atom count configured in `shared.source`.

mmml md-system cutoff schematics (steps 16–19)

These scripts call `mmml md-system` with different ensembles. Logs record a complementary cutoff schematic PNG; copies are placed in `assets/cli/` when the generator finds the source files.

```
bash 16_md_10mer_free_nve.sh
bash 17_md_10mer_free_nvt.sh
bash 18_md_10mer_pbc_nve.sh
bash 19_md_10mer_pbc_nvt.sh
```

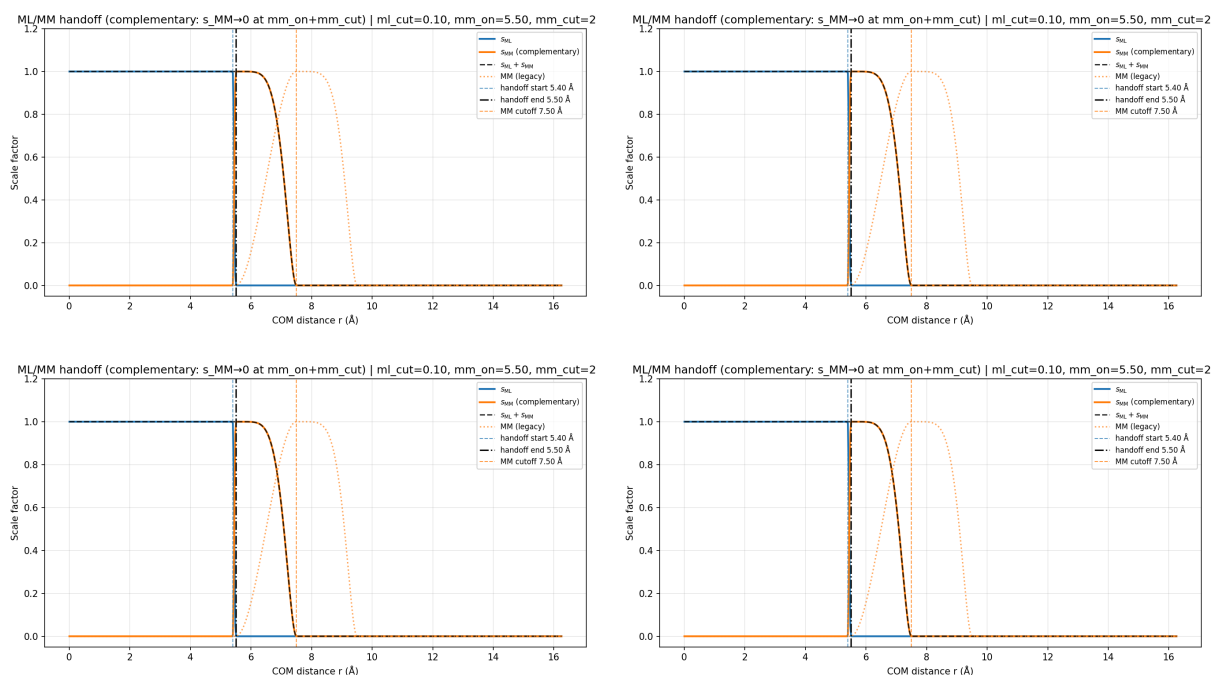



Figure 14: Cutoff layout figures referenced from 16.log–19.log (paths may differ on your machine).

Periodic NPT and mixed-solvent examples: 20_md_10mer_pbc_npt.sh, 21_md_system_meoh_tip3_1to1.sh. The md-system wrapper has an explicit --backend auto|ase|jaxmd selector: auto keeps ASE for the free/PBC NVE/NVT presets and routes pbc_npt through JAX-MD. Use --backend jaxmd when you want the periodic NVE/NVT presets to use the JAX-MD runner too.

Periodic presets choose a cubic box automatically from the initial cluster extent plus padding. Override it with --box-size <angstrom> or, in the tutorial shell scripts, with MDSYS_BOX_A:

```
MDSYS_PS=10 MDSYS_N_MOLECULES=100 MDSYS_BOX_A=60 bash 20_md_10mer_pbc_npt.sh
```

JAX-MD preset runs now do a staged pre-MD minimization. CHARMM SD/ABNR runs first, including an energy/force warmup with the internal force-field terms active. The MMML calculator then runs ASE BFGS to $f_{\max}=0.1$; if BFGS does not reach that threshold, ASE FIRE is used before the JAX-MD minimizer starts. The optimizer target remains 0.1, while the default abort threshold is looser at --max-fmax-after-min 0.5. Tune these lower-level knobs by putting --extra-args last, for example:

```
mmml md-system --setup pbc_npt --backend jaxmd --extra-args --pre-min-steps 200 --fire-min-steps 500
```

Optional scripts (XX_*)

- XX_pyscf_dft_cli.sh — minimal DFT energy demo.
- XX_pyscf_mp2_cli.sh — MP2 reference example.

shared.source variables you will likely edit

- RES, PHYSNET_TRAINER, PHYSNET_CKPT_DIR, PHYSNET_NAME, PHYSNET_CHECKPOINT
- PHYSNET_NATOMS, PHYSNET_EPOCHS, PHYSNET_BATCH
- PHYSNET_TRAIN_NPZ, PHYSNET_VALID_NPZ
- PHYSNET_MD_OUT, PHYSNET_MD_TRAJ, ACTIVE_LEARNING_OUT
- MDSYS_* knobs for md-system demos

Fast start options

Option A: bundled full script

```
cd mmml_tutorial/cli
bash run_full_training.sh
```

(Still runs expensive QM; shorten epochs/checkpoints inside the script for demos.)

Option B: manual progression

```
cd mmml_tutorial/cli
bash 01_make_res_cli.sh
bash 02_make_box_cli.sh
bash 04_pyscf_dft_cli_full.sh
bash 06_normal_mode_sample_cli.sh
bash 07_pyscf_evaluate_cli.sh
bash 08_fix_and_split_cli.sh
bash 10_physnet_dcmnet_train_cli.sh
```

Option C: joint smoke test (splits prepared)

```
cd mmml_tutorial/cli
JAX_PLATFORMS=cpu uv run python -m mmml.cli.misc.train_joint \
  --train-efd out/splits/energies_forces_dipoles_train.npz \
  --train-esp out/splits/grids_esp_train.npz \
  --valid-efd out/splits/energies_forces_dipoles_valid.npz \
  --valid-esp out/splits/grids_esp_valid.npz \
  --use-repo-physnet-params \
  --epochs 1 \
  --batch-size 1 \
  --name smoke_joint \
  --ckpt-dir /tmp/mmml_ckpts \
  --plot-freq 0
```

Caveats

- Script echo lines are sometimes shortened; the shell body is authoritative.
- pyscf-evaluate may append _2, _3, ... when outputs exist; step 08 always consumes the newest file.
- GPU JAX failures before argparse: prefix with JAX_PLATFORMS=cpu.
- Portable PhysNet JSON must match the architecture expected by train_joint for your system size.

Troubleshooting

Joint training parameter merge errors

Healthy startup usually prints:

```
Loaded PhysNet config from checkpoint
Merging PhysNet params into joint model
Pre-computing edge lists
```

PySCF or CHARMM too slow for live sessions

Precompute out/04_results.h5, out/06_sampled.npz, out/07_evaluated*.npz, and out/splits/*.npz, then teach from logs + figures and run only short training.

Appendix: water-dimer or other dedicated layouts

Some course materials use a water-dimer directory with out/splits_dimer/ and different train_joint names. The workflow is the same at the CLI level; only paths and NATOMS change. Start from the appropriate shared.source overrides for that dataset.

Next additions

- Wall-time table on reference hardware.
- Screenshots from mmml gui on trajectories produced in step 13.